

Seguridad Informática - Práctica 2 – Algoritmo RSA

Mauricio A Sonda Cahuich - 66611

5 de septiembre de 2026

1. ¿Qué hice?

Para esta práctica programé en Python el algoritmo RSA para descifrar un mensaje que nos dieron, usando los primos $p = 67$ y $q = 83$. La idea era calcular n , $\varphi(n)$, encontrar el cuarto valor de e que sirviera como clave pública, sacar su d correspondiente (clave privada) y con eso descifrar el mensaje.

Lo que hice diferente fue no solo contar los candidatos de e hasta llegar al cuarto, sino que hice que el programa **probara cada uno** intentando descifrar el mensaje real, para comprobar cuál de verdad funciona y así tener una justificación más sólida (y no solo "porque sí es el cuarto").

2. Datos de inicio

$$p = 67$$

$$q = 83$$

Mensaje cifrado:

1058, 2597, 4955, 4201, 3162, 4343, 2754, 2497, 336, 2597

3. Paso 1: Sacar n

Esto es lo más fácil, nada más se multiplican los primos:

$$n = p \times q = 67 \times 83 = 5561$$

4. Paso 2: Sacar $\varphi(n)$

$$\varphi(n) = (p - 1)(q - 1) = 66 \times 82 = 5412$$

En el código nada más los calculo y los imprimo para tener registro:

```
1 n = p * q
2 phi = (p - 1) * (q - 1)
3
4 print("n = " + str(n))
5 print("phi(n) = " + str(phi) + "\n")
```

5. Paso 3: Buscar el e correcto

Aquí es donde le puse más trabajo. Sabemos que e tiene que cumplir:

$$1 < e < \varphi(n) \quad \text{y} \quad \text{mcd}(e, \varphi(n)) = 1$$

Entonces hice un while que va probando números desde el 2 hacia arriba, y cada vez que encuentra uno que cumple lo del mcd, lo cuenta como candidato. Pero en vez de solo quedarme con el cuarto y ya, hice que el programa **intentara descifrar el mensaje con cada candidato**, y si el resultado eran puros caracteres raros/no imprimibles, lo descartaba y seguía con el siguiente.

```
1 e = 2
2 while e < phi and not encontrado:
3
4     if mcd(e, phi) != 1:
5         e += 1
6         continue # no cumple, ni lo cuento
7
8     contador += 1
9     d = calcularD(e, phi)
10
11     print("Candidato #" + str(contador) + ": e = " + str(e) + ", d = " + str(d))
12
13     intento = descifrarMensaje(mensajeCifrado, d, n)
14
15     if esTextoValido(intento):
16         print("Este si funciona! usamos e = " + str(e))
17         encontrado = True
18     else:
19         print("Este no sirve, el texto sale con basura.")
20
21     e += 1
```

Para saber si el texto "sirve" hice una función que checa que todos los caracteres estén en el rango normal de ASCII imprimible (del 32 al 126, o sea letras, números y símbolos normales):

```
1 def esTextoValido(texto):
2     for c in texto:
3         if ord(c) < 32 or ord(c) > 126:
4             return False # salio un caracter raro, este e no sirve
5     return True
```

5.1. ¿Por qué el cuarto candidato es el correcto?

Al correr el programa, los primeros candidatos que cumplen lo del mcd son 5, 7, 13 y 17. Cuando los probé uno por uno descifrando el mensaje:

- Con $e = 5$ el mensaje sale puro símbolo raro, no sirve.
- Con $e = 7$ pasa lo mismo, sale basura.
- Con $e = 13$ otra vez sale texto sin sentido.
- Con $e = 17$ (el cuarto) el mensaje **sí sale legible**, con letras normales.

Entonces ahí queda comprobado: $e = 17$ no solo es el cuarto número que cumple la condición matemática, además es el que de verdad descifra bien el mensaje. Eso es la prueba de que es el correcto.

6. Paso 4: Sacar d (clave privada)

d tiene que cumplir que:

$$e \times d \equiv 1 \pmod{\varphi(n)}$$

En vez de usar el algoritmo extendido de Euclides a mano, hice que el programa lo fuera probando con un while hasta encontrar el valor que cumpliera:

```
1 def calcularD(e, phi):
2     d = 1
3     while (d * e) % phi != 1:
4         d += 1 # le sigo subiendo hasta que de resto 1
5     return d
```

7. Paso 5: Descifrar el mensaje

Para descifrar cada número del mensaje se usa la fórmula:

$$m = c^d \pmod{n}$$

Lo hice con un for que va multiplicando y sacando el módulo poco a poco (en vez de elevar todo de un jalón, porque los números se hacen enormes):

```
1 def descifrar(c, d, n):
2     m = 1
3     # aquí se hace (c elevado a la d) mod n, pero poco a poco
4     for i in range(d):
5         m = (m * c) % n
6     return chr(m)
```

8. Resultados

- **Clave pública:** $(e, n) = (17, 5561)$
- **Clave privada:** (d, n) , la calcula el programa
- **Mensaje cifrado:** 1058, 2597, 4955, 4201, 3162, 4343, 2754, 2497, 336, 2597
- **Mensaje descifrado:** HOLA MUNDO

9. Evidencias

- **Evidencia:** Captura con el resultado final en terminal

[illegible]

Figura 1: Consola mostrando el calculo e intentos.

10. Conclusión

Con este programa no nada más se sacan las claves y se descifra el mensaje, sino que traté de hacer visible de manera clara cómo se llega al resultado final, y el porque el cuarto candidato de e es el correcto: se prueban los candidatos válidos hasta que uno logra descifrar el mensaje de forma legible. Eso deja comprobado, con evidencia del propio programa, que la elección de $e = 17$ es la correcta.